# C-346 — C Programming Fundamentals & Data Structures

## COMPREHENSIVE REFERENCE MATERIALS & CHEAT SHEET

| | |
|---|---|
| **Course Code:** C-346 | **Category:** Computer Science & Tech |
| **Academic Term:** 2026 Academic Year | **Format:** E-Learning / Self-Paced |
| **Prerequisites:** Basic Problem Solving | **Estimated Hours:** 40 Hours Content |

## 1. Core Concepts & Theoretical Framework

This reference document provides quick access to core syntax, APIs, design principles, and best practices for **C Programming Fundamentals & Data Structures**. Keep this sheet accessible as you work through lab exercises and projects.

### Fundamental Rules & Design Conventions:

1. **Clean & Readable Code:** Prioritize clear variable naming, modular design, and concise function responsibilities.
2. **Error Handling & Robustness:** Always validate inputs, handle edge cases, and catch runtime exceptions gracefully.
3. **Efficiency & Complexity:** Pay attention to Time Complexity O(N) and Space Complexity constraints.
4. **Documentation:** Maintain inline docstrings, type annotations, and README instructions.

## 2. Quick Reference Syntax & Code Templates

```
# Example 1: Standard Initialization & Workflow def process_course_data(records: list) -> dict: results = {"total": len(records), "valid": 0, "processed": []} for record in records: if record.get("status") == "active": results["valid"] += 1 results["processed"].append(record["id"]) return results

// Example 2: Asynchronous Execution & API Handling async function fetchCourseDetails(courseId) { try { const response = await fetch(`/api/courses/${courseId}`); if (!response.ok) throw new Error(`HTTP Error: ${response.status}`); const data = await response.json(); return data.course; } catch (error) { console.error("Failed to load course details:", error); return null; } }
```

## 3. Recommended Reading & Official Documentation

- **Official Documentation:** Official language specifications, API docs, and standard library guides.
- **Recommended Textbook:** *Clean Code: A Handbook of Agile Software Craftsmanship*
- **Design Patterns:** *Elements of Reusable Object-Oriented Software*
- **Community Resources:** Developer Hubs, StackOverflow, and GitHub repositories.